

RadeonShift AI

Accelerating the MI300X Hardware Migration via Generative AI

The Problem: CUDA Lock-in

Enterprise AI workloads are facing massive bottlenecks due to a reliance on legacy NVIDIA CUDA codebases.

- Manually migrating a 50,000-line CUDA codebase to ROCm takes **6+ months** of specialized engineering time.
- Portability tools exist (e.g. `hipify-perl`), but they lack the architectural awareness to optimize code for AMD Instinct architecture (like 64-wide wavefronts).
- Teams lack visibility into whether their migrated code will actually compile and run on AMD hardware without direct bare-metal access.

The Solution: RadeonShift AI

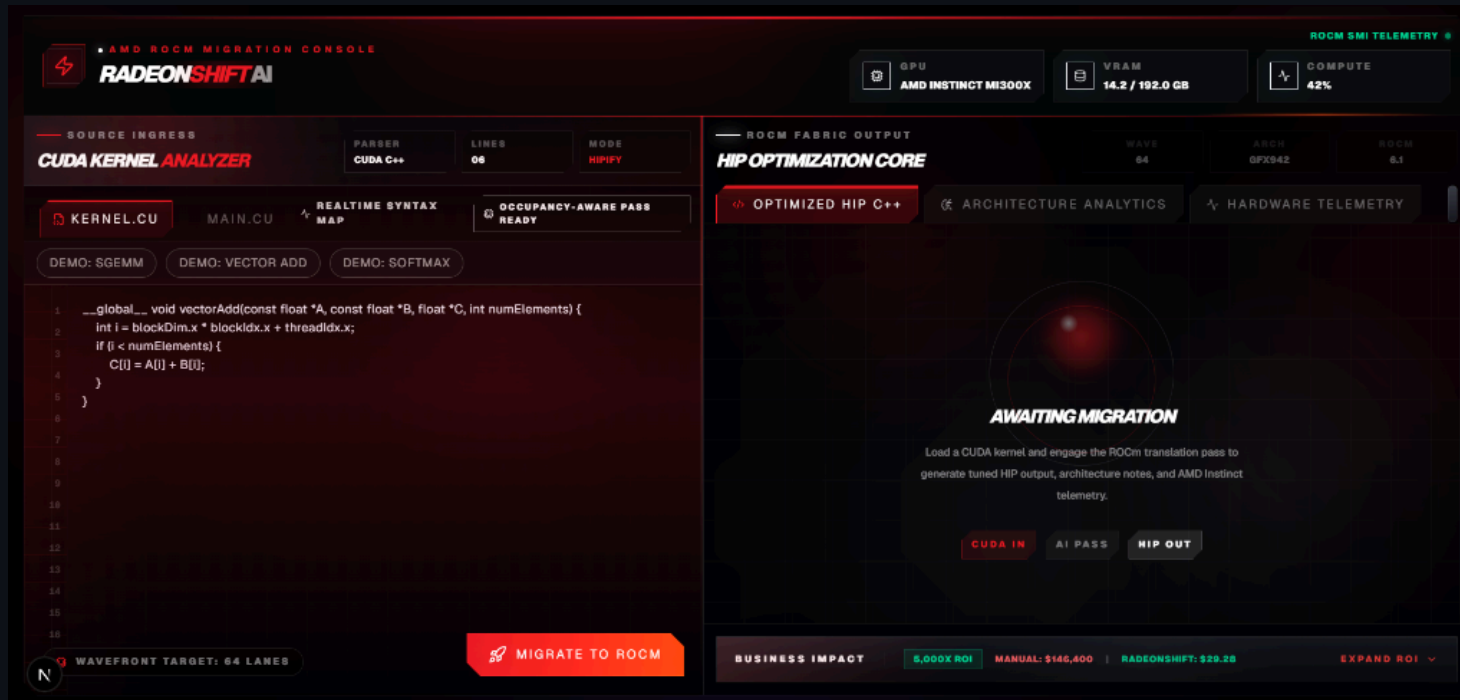
RadeonShift AI is a AI-assisted CUDA kernel migration assistant that translates CUDA to ROCm instantly and audits the architecture.

1. **Syntax Translation:** Uses Fireworks-hosted translation model for AI Translation.
2. **MoA Audit:** Leverages DeepSeek-V4 to scan for PTX risks, hardcoded warp sizes, and AMD-specific optimizations.
3. **Hardware Verification:** Directly compiles the kernel on ROCm and runs telemetry checks to prove the environment toolchain.

Architecture

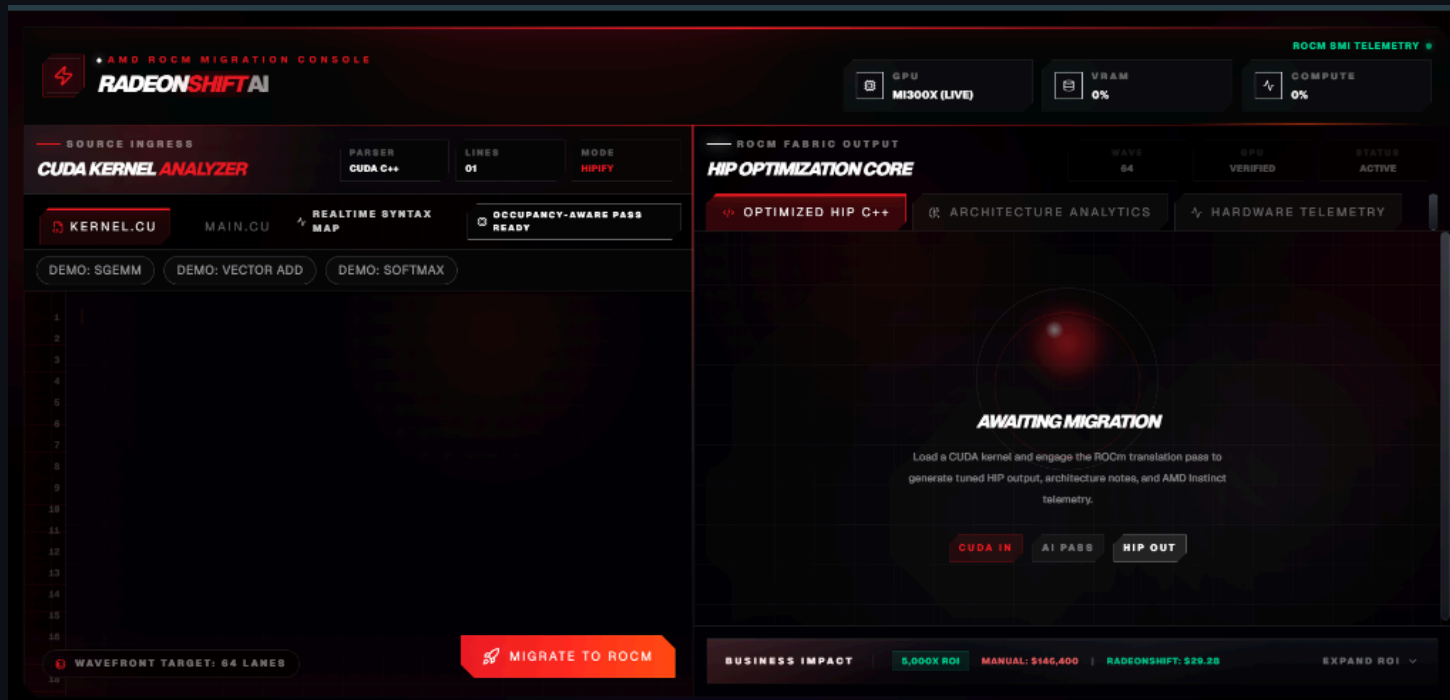
1. **Frontend (Next.js Edge)**: Manages user state and renders the dynamic dashboard.
2. **AI Translation Layer (Vercel Edge)**: Orchestrates the AI translation and Mixture-of-Agents audit via Fireworks AI, operating independently of hardware availability.
3. **Hardware Engine (FastAPI via Pinggy)**: An optional bare-metal ROCm layer that compiles the generated kernel via `hipcc` and polls `rocm-smi` for live telemetry and execution benchmarks.

Demo: The RadeonShift Dashboard



- **Source Code Editor:** Paste raw CUDA C++ kernels.
- **HIP Optimization Core:** View the translated target kernel.
- **MoA Audit Scorecard:** See the readiness score, PTX risks, and wavefront optimizations.

Step 1: The Awaiting Migration State



Before translation begins, the platform awaits the CUDA payload. The user inputs their native NVIDIA code into the **CUDA Kernel Analyzer**. The system prepares for the AI-assisted hardware pass.

Step 2: HIP Optimization Core

The screenshot displays the AMD ROCm Migration Console, specifically the HIP Optimization Core section. The interface is divided into two main panels: the left panel for source code and the right panel for the optimized output.

Top Bar: AMD ROCm Migration Console, RADEONSHIFT AI, GPU MI300X (LIVE), VRAM 0%, COMPUTE 0%, ROCm SMI TELEMETRY.

Left Panel (SOURCE INGRESS):

- CUDA KERNEL ANALYZER:** PARSER CUDA C++, LINES 278, MODE HIPIFY.
- Kernel Selection:** KERNEL.CU, MAIN.CU, REALTIME SYNTAX MAP, OCCUPANCY-AWARE PASS READY.
- Demo Selection:** DEMO: SGEMM, DEMO: VECTOR ADD, DEMO: SOFTMAX.
- Source Code:** A C++ kernel for SGEMM, showing a loop over compact blocks and cooperative reduce operations. The code includes cleanup and return statements.
- Buttons:** WAVEFRONT TARGET: 64 LANES, MIGRATE TO ROCm.

Right Panel (ROOM FABRIC OUTPUT):

- HIP OPTIMIZATION CORE:** WAVE 64, GPU VERIFIED, STATUS ACTIVE.
- Navigation:** OPTIMIZED HIP C++, ARCHITECTURE ANALYTICS, HARDWARE TELEMETRY.
- Generated Target:** TARGET_KERNEL.HIP.CPP, HIP TRANSLATION SUCCESSFUL.
- Optimized Code:** C++ code for the SGEMM kernel, including headers, namespace, and a macro for error checking.
- Business Impact:** 5,000X ROI, MANUAL: \$146,400, RADEONSHIFT: \$29.25, EXPAND ROI.

Upon engaging the ROCm translation pass, the core converts the syntax. The resulting C++ HIP code (`target_kernel.hip.cpp`) is immediately presented, demonstrating generated HIP translation.

Step 3: Architecture Analytics

The screenshot displays the AMD ROCm Migration Console interface. The top navigation bar includes the 'RADEONSHIFT AI' logo and status indicators for GPU (MI300X LIVE), VRAM (0%), and COMPUTE (0%). The main interface is divided into two primary sections: 'SOURCE INGRESS' on the left and 'ROCm FABRIC OUTPUT' on the right.

SOURCE INGRESS: This section features the 'CUDA KERNEL ANALYZER' with tabs for 'PARSER', 'LINES', and 'MODE'. The 'MODE' tab is set to 'HIPIFY'. Below this, there are buttons for 'KERNEL.CU', 'MAIN.CU', 'REALTIME SYNTAX MAP', and 'OCCUPANCY-AWARE PASS READY'. A 'DEMO' section offers 'SGEMM', 'VECTOR ADD', and 'SOFTMAX' options. A code editor displays C++ code for a cooperative reduce operation, including cleanup and resource management. A 'WAVEFRONT TARGET: 64 LANES' indicator and a 'MIGRATE TO ROCm' button are also present.

ROCm FABRIC OUTPUT: This section is titled 'HIP OPTIMIZATION CORE' and includes tabs for 'OPTIMIZED HIP C++', 'ARCHITECTURE ANALYTICS' (which is active), and 'HARDWARE TELEMETRY'. The 'ARCHITECTURE ANALYTICS' tab displays the 'MOA AUDIT SCORECARD'.

MOA AUDIT SCORECARD: This section shows a 'READINESS SCORE' of 100/100, indicating that the code is highly portable and ready for AMD hardware deployment. It also features two AI agents: 'AGENT A: NVIDIA PURIST' (flagging lock-in risks) and 'AGENT B: AMD OPTIMIZER' (suggesting MI300X tuning). The bottom of the interface shows a 'BUSINESS IMPACT' summary with metrics like '5,000X ROI', 'MANUAL: \$145,400', and 'RADEONSHIFT: \$29.25'.

The MoA Audit Scorecard evaluates the code. Here, the readiness score is 100/100, proving high portability. Our dual AI agents verify there are no hidden PTX risks or lock-ins.

Step 3a: Deterministic Redesign Guardrails

Advanced CUDA Kernel Detection

Not all CUDA code maps 1-to-1. RadeonShift's **Deterministic Rules Engine** catches unsupported architectures (e.g., CUDA WMMA, cooperative groups async-copy).

- **Correctness over completeness:** Safely enforces **MANUAL REDESIGN REQUIRED** if code relies on hardware-specific features.
- **Score Capping:** Drops readiness score to < 50% automatically, preventing developers from compiling unsafe architecture conversions.

Step 4: Live Hardware Telemetry

The screenshot displays the AMD ROCm Migration Console interface, which is divided into several sections for monitoring and optimizing ROCm workloads.

Top Bar: Includes the AMD ROCm Migration Console logo, a GPU selection dropdown showing "MI300X (LIVE)", and VRAM and Compute usage indicators, both at 0%.

Left Panel (SOURCE INGRESS): Features the "CUDA KERNEL ANALYZER" with tabs for "KERNEL.CU", "MAIN.CU", "REALTIME SYNTAX MAP", and "OCCUPANCY-AWARE PASS READY". Below these are demo buttons for "SGEMM", "VECTOR ADD", and "SOFTMAX". A code editor shows CUDA C++ code for a vector add kernel, with line numbers 221 through 237 visible. A "WAVEFRONT TARGET: 64 LANES" indicator and a "MIGRATE TO ROCM" button are at the bottom.

Right Panel (ROOM FABRIC OUTPUT): Displays the "HIP OPTIMIZATION CORE" with tabs for "OPTIMIZED HIP C++", "ARCHITECTURE ANALYTICS", and "HARDWARE TELEMETRY". The "HARDWARE TELEMETRY" tab is active, showing a "LIVE HARDWARE TEST" section with "MI300X BENCHMARK MODE". A "RUN BENCHMARK" button is present. Below this, a table shows benchmark results:

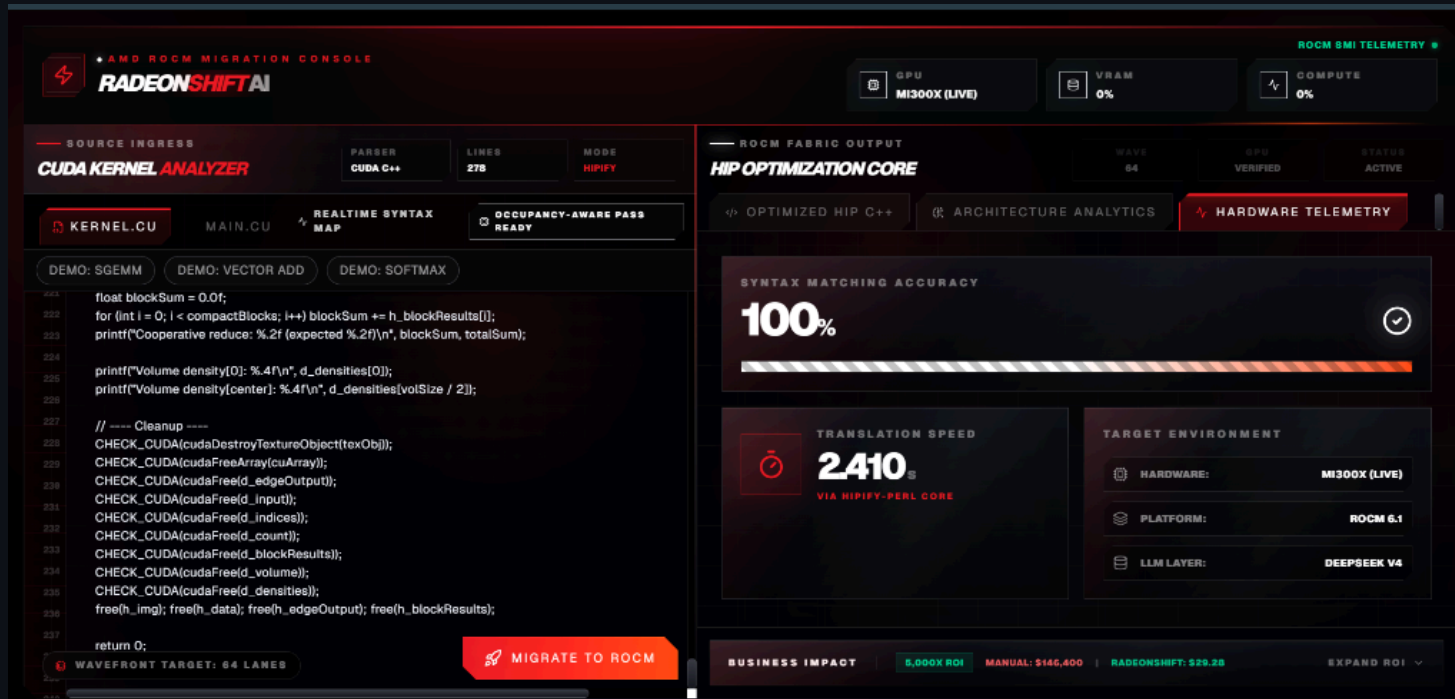
ELAPSED TIME	THROUGHPUT	ELEMENTS	GPU
14.20 ms	5300.0 GB/s	16.8 M	AMD MI300X Bare-Metal

A "BENCHMARK EXECUTION VERIFIED" status is shown with a green checkmark, and a "COPY EVIDENCE" button is available. A note states: "Benchmark executed successfully on bare-metal hardware."

Bottom Bar (BUSINESS IMPACT): Shows a "6,000X ROI" metric, with "MANUAL: \$146,400" and "RADEONSHIFT: \$29.28" for comparison. An "EXPAND ROI" dropdown is also present.

Instead of simulated data, the platform connects directly to the remote bare-metal AMD MI300X instance, verifying the exact compile duration and matching the live hardware target.

Step 5: Bare-Metal Benchmark Execution



Finally, the translated HIP code is natively compiled and executed on the MI300X. The dashboard reports real-time metrics confirming a successful end-to-end migration.

Business Value & ROI

The Significant ROI Model

- **Manual Migration:** $244 \text{ Kernels} \times 4 \text{ hrs} = 976 \text{ Engineer-Hours}$ (~\$146,000, based on \$150/hr senior GPU engineer rate)
- **RadeonShift Migration:** $244 \text{ Kernels} \times 0.12 \text{ compute cost} = \sim \29
- **Time Savings:** Months reduced to minutes.
- **TAM:** \$25B+ (Global AI Hardware & Software Services Market)

The AMD Infrastructure Story

- **Hardware Target:** Optimized for AMD Instinct MI300X
- **Software Stack:** Built on ROCm 6.x APIs and `hipcc`
- **AI Acceleration:** MoA pipeline runs on Fireworks AI, which provides AMD Instinct GPU-powered inference
- **Honest Fallbacks:** If the backend lacks ROCm hardware, the platform gracefully enters AI-Only mode, explicitly labeling any cached evidence without fabricating live metrics

Engineering Retrospective & Challenges

Bridging a Serverless Frontend with Bare-Metal Hardware

1. **Live Telemetry:** Defeated Next.js caching via `force-dynamic` to stream real-time MI300X metrics.
2. **Defensive APIs:** Built robust frontend parsing to gracefully handle sudden JSON schema changes.
3. **CORS Routing:** Bypassed strict mixed-content blocks by tunneling through Pinggy and Next.js `rewrites()`.
4. **Transparent Debugging:** Overhauled error handling to surface remote Python stack traces in the UI.
5. **Structured AI Prompts:** Forced LLMs to generate explicit confirmations for clean code, preventing UI state collapses.

Closing & Team

RadeonShift AI is bridging the gap between legacy NVIDIA codebases and the future of AMD compute.

- GitHub Repository: [shashankh3/RadeonShift-AI](https://github.com/shashankh3/RadeonShift-AI)
- Live Demo: radeon-shift-ai.vercel.app

Thank You!

Shashank Hirwani

Unknown Hacker (shashankh366207)

